

SecureSandbox - Custom Process Isolation System

Complete Project Documentation

Table of Contents

1. Project Overview
 2. Features
 3. Installation
 4. Usage Guide
 5. Technical Implementation
 6. Test Programs
 7. Security Features
 8. Future Enhancements
-

1. Project Overview

What is SecureSandbox?

SecureSandbox is a custom-built process isolation and monitoring system designed for secure execution of untrusted programs. It provides a controlled environment where programs can run with limited access to system resources while their behavior is continuously monitored.

Purpose

- **Malware Analysis:** Safely execute and analyze potentially malicious programs
- **Secure Testing:** Test unknown code without risking system security
- **Process Monitoring:** Track resource usage and system calls
- **Educational:** Learn about process isolation and OS security

Cyber Security Relevance

- Process isolation techniques
- Memory access control
- System call monitoring
- Resource limitation
- Behavioral analysis

2. Features

Core Features

Process Isolation

- Restricted environment variables
- Isolated home and temp directories
- Limited file system access
- Blocked system directories

Resource Monitoring

- Real-time CPU usage tracking
- Memory consumption monitoring
- Thread count tracking
- Network connection monitoring

Activity Logging

- Detailed execution logs
- File access tracking
- Network activity logging
- Security violation detection

Security Controls

- Path restriction enforcement
- Process termination capability
- Resource usage alerts
- Suspicious behavior detection

Reporting

- JSON export of execution data
- Resource usage graphs
- Complete activity timeline
- Security assessment report

3. Installation

Requirements

```
bash

Python 3.7+
tkinter (usually included with Python)
psutil
```

Installation Steps

Step 1: Install Python

- Download from python.org
- Version 3.7 or higher required

Step 2: Install Dependencies

```
bash

pip install psutil
```

Step 3: Download SecureSandbox

- Save the main code as `secure_sandbox.py`
- Save test programs in a `test_programs/` folder

Step 4: Verify Installation

```
bash

python secure_sandbox.py
```

Directory Structure

```
SecureSandbox/
├── secure_sandbox.py      # Main application
├── test_programs/        # Test scripts
│   ├── test_normal.py
│   ├── test_resources.py
│   ├── test_file_access.py
│   ├── test_network.py
│   └── test_suspicious.py
```

```
| |— test_infinite.py
| |— test_multithreaded.py
| |— test_memory.py
|— sandbox_home/      # Isolated home (auto-created)
|— sandbox_temp/      # Isolated temp (auto-created)
|— reports/           # Generated reports
```

4. Usage Guide

Basic Workflow

Step 1: Launch Sandbox

```
bash

python secure_sandbox.py
```

Step 2: Select Program

- Click "Browse" button
- Select the program to execute
- Supported: Python files (.py), executables (.exe)

Step 3: Add Arguments (Optional)

- Enter command-line arguments in Arguments field
- Example: `--verbose --output result.txt`

Step 4: Start Execution

- Click "▶ Start Execution"
- Monitor real-time activity in tabs

Step 5: Monitor Execution

- **Activity Logs:** View all system events
- **Resource Monitor:** Track CPU, memory, threads
- **Program Output:** See stdout/stderr

Step 6: Stop if Needed

- Click "■ Stop Execution" to terminate
- Process will be forcefully killed if necessary

Step 7: Generate Report

- Click " Generate Report"
- JSON file created with complete analysis

Understanding the Interface

Activity Logs Tab

Color-coded log entries:

- **Blue (INFO):** General information
- **Green (SUCCESS):** Successful operations
- **Yellow (WARNING):** Warnings
- **Red (ERROR):** Errors
- **Dark Red (BLOCKED):** Blocked security violations
- **Purple (FILE):** File access events
- **Yellow (NETWORK):** Network activities

Resource Monitor Tab

Real-time tracking:

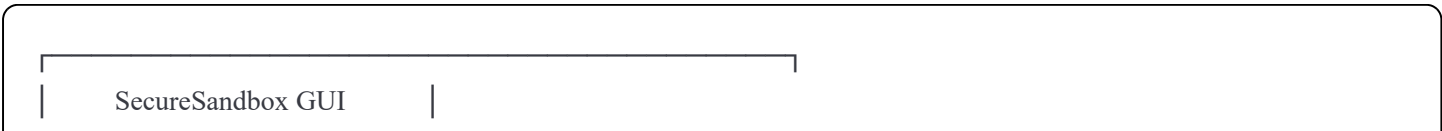
- **Time:** Timestamp
- **CPU%:** CPU usage percentage
- **Memory(MB):** RAM consumption
- **Threads:** Active thread count

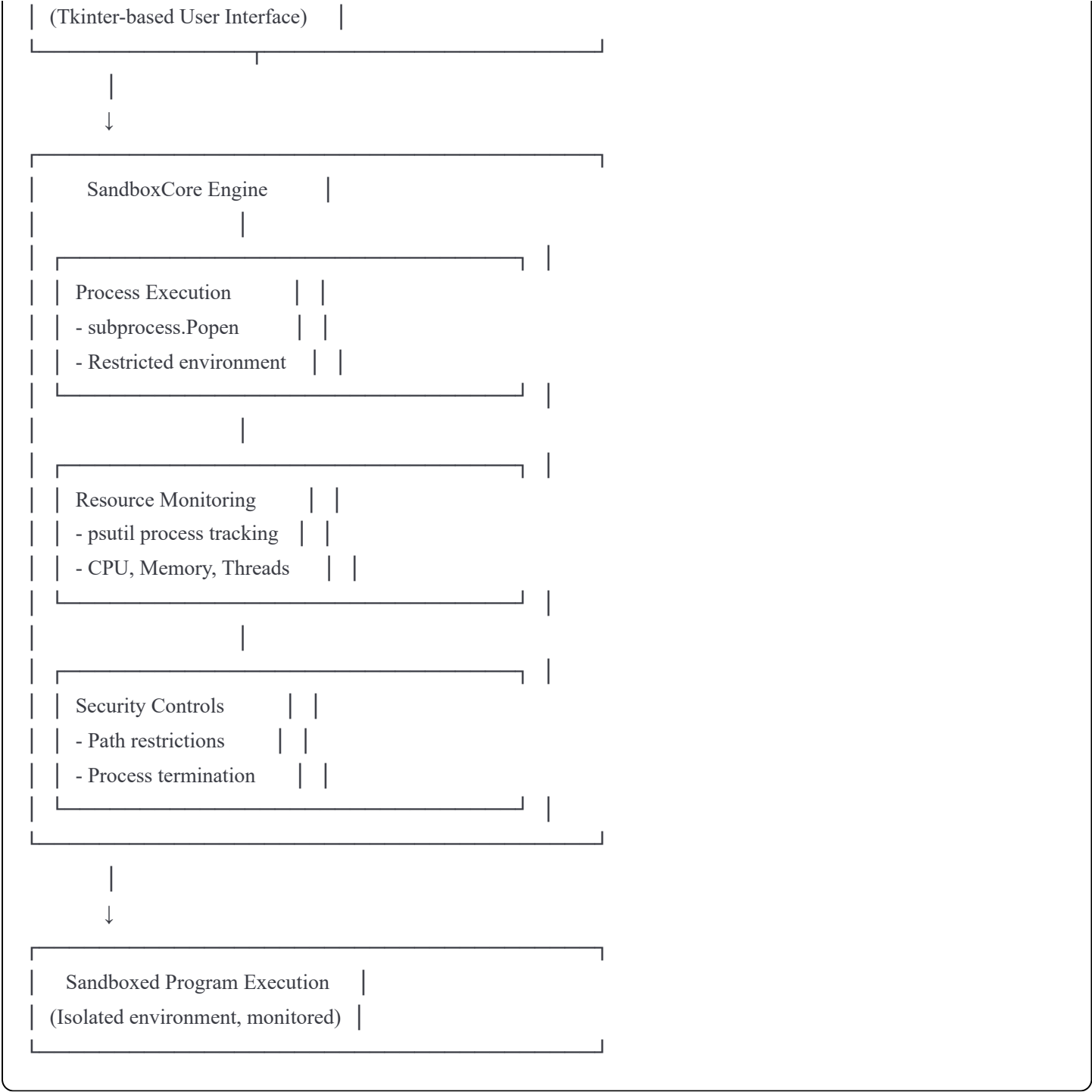
Program Output Tab

- Standard output (stdout)
- Error output (stderr)
- Real-time display

5. Technical Implementation

Architecture Overview





Key Components

1. SandboxCore Class

Handles core sandboxing functionality:

- Environment isolation
- Process execution
- Resource monitoring
- Security enforcement

2. Environment Isolation

```
python
```

```
env = os.environ.copy()
env['HOME'] = 'sandbox_home'
env['TEMP'] = 'sandbox_temp'
```

Creates isolated directories for program execution

3. Process Monitoring

Uses `psutil` to track:

- CPU usage: `proc.cpu_percent()`
- Memory: `proc.memory_info().rss`
- Threads: `proc.num_threads()`
- Network: `proc.connections()`
- Files: `proc.open_files()`

4. Path Restrictions

Blocked paths:

- User home directory
- System directories (Windows System32, /etc, /sys, /proc)
- Custom additions possible

5. Logging System

Structured logging with:

- Timestamp
 - Severity level
 - Detailed message
 - JSON export capability
-

6. Test Programs

Test Program Descriptions

1. test_normal.py

Purpose: Basic functionality test **What it does:**

- Prints messages
- Simple loop with delays
- Normal program completion

Expected behavior:

- Clean execution
 - No security violations
 - Low resource usage
-

2. test_resources.py

Purpose: Resource consumption test **What it does:**

- Allocates increasing memory
- CPU-intensive calculations
- Stress testing

Expected behavior:

- High CPU and memory usage
 - Multiple resource usage spikes
 - Shows sandbox monitoring capabilities
-

3. test_file_access.py

Purpose: File system access test **What it does:**

- Creates files in sandbox
- Reads/writes data
- Attempts restricted path access

Expected behavior:

- Sandbox file operations allowed
 - Restricted path access blocked
 - File access logged
-

4. test_network.py

Purpose: Network activity test **What it does:**

- Creates socket connection
- Connects to external server
- Sends/receives data

Expected behavior:

- Network connections logged
 - Connection details tracked
 - HTTP request/response
-

5. test_suspicious.py

Purpose: Malicious behavior simulation **What it does:**

- System directory access attempts
- Child process spawning
- Environment modification
- High CPU usage

Expected behavior:

- Multiple security violations logged
 - Blocked activities detected
 - Behavioral analysis data
-

6. test_infinite.py

Purpose: Process termination test **What it does:**

- Runs infinite loop
- Tests stop functionality

Expected behavior:

- Continuous execution until stopped
- Clean termination on stop button

- Resource tracking throughout
-

7. test_multithreaded.py

Purpose: Thread monitoring test **What it does:**

- Creates multiple threads
- Concurrent execution
- Thread synchronization

Expected behavior:

- Thread count increases
 - Multiple concurrent activities
 - Resource sharing logged
-

8. test_memory.py

Purpose: Memory allocation test **What it does:**

- Allocates large memory blocks
- Tests memory tracking
- Object lifecycle management

Expected behavior:

- Memory usage increases
 - Peak memory tracked
 - Garbage collection monitored
-

7. Security Features

Implemented Security Controls

1. Environment Isolation

- Separate HOME directory
- Isolated TEMP directory
- Custom environment variables

- Prevents system-wide changes

2. File System Restrictions

Blocked Paths:

python

User home directory

C:\Windows\System32 (Windows)

/etc (Linux)

/sys (Linux)

/proc (Linux)

3. Resource Limitations

- CPU usage monitoring
- Memory consumption tracking
- Thread count limits
- Network activity monitoring

4. Process Control

- Graceful termination
- Forceful kill if needed
- Parent process isolation
- No privilege escalation

5. Activity Logging

- All file access logged
- Network connections tracked
- Process creation monitored
- Security violations flagged

Security Considerations

What SecureSandbox DOES:  Monitor program behavior  Restrict file system access  Track resource usage  Log all activities  Terminate malicious processes

What SecureSandbox DOES NOT:  Prevent all attacks (not enterprise-grade)  Provide kernel-level isolation  Guarantee 100% security  Replace professional sandboxes (Cuckoo, etc.)  Protect against sophisticated exploits

Recommended Usage:

- Educational purposes
- Initial malware analysis
- Testing unknown code
- Learning security concepts
- Research and development

NOT recommended for:

- Production malware analysis
 - Critical infrastructure
 - Highly sensitive environments
 - Advanced persistent threats
 - Zero-day exploit analysis
-

8. Future Enhancements

Planned Features

1. Advanced Isolation

- Docker container integration
- Virtual machine support
- Kernel-level restrictions
- Namespace isolation

2. Enhanced Monitoring

- System call tracing (strace integration)
- Registry monitoring (Windows)
- Kernel module detection
- DLL injection detection

3. Network Analysis

- Packet capture (pcap)
- DNS query logging
- HTTP/HTTPS traffic analysis

- Protocol detection

4. Behavioral Analysis

- API call pattern recognition
- Heuristic detection
- Machine learning classification
- Threat scoring

5. Reporting

- PDF report generation
- Graphical charts
- Timeline visualization
- Comparison reports

6. User Interface

- Web-based dashboard
 - Real-time graphs
 - Alert notifications
 - Custom rule creation
-

9. Usage Examples

Example 1: Testing Normal Program

bash

1. Open SecureSandbox
2. Browse → test_programs/test_normal.py
3. Click "Start Execution"
4. Observe: Clean execution, low resources
5. Generate Report

Example 2: Analyzing Suspicious Behavior

bash

1. Load test_suspicious.py
2. Start execution
3. Watch Activity Logs for BLOCKED entries
4. Check Resource Monitor for spikes
5. Review generated report for security violations

Example 3: Resource Stress Test

bash

1. Load test_resources.py
2. Monitor Resource Monitor tab
3. Observe CPU and Memory increases
4. Compare against baseline

10. Troubleshooting

Common Issues

Issue: Program doesn't start **Solution:**

- Check file path is correct
- Verify Python interpreter
- Check file permissions

Issue: No resource data shown **Solution:**

- Wait 1-2 seconds for data collection
- Ensure program is running
- Check psutil installation

Issue: Can't stop program **Solution:**

- Click Stop button twice
- Wait for forceful kill
- Close sandbox and restart

Issue: Permission denied errors **Solution:**

- Run as administrator (if needed)
- Check file access permissions
- Verify sandbox directories created

11. Project Demonstration

For Project Presentation

Demo Flow:

1. Introduction (2 min)

- Explain purpose
- Show interface

2. Normal Execution (3 min)

- Run test_normal.py
- Show monitoring features
- Display logs

3. Security Test (5 min)

- Run test_suspicious.py
- Highlight blocked activities
- Explain security features

4. Resource Monitoring (3 min)

- Run test_resources.py
- Show CPU/Memory graphs
- Explain tracking

5. Report Generation (2 min)

- Generate JSON report
- Show report contents
- Explain analysis

Key Points to Highlight:

- Real-time monitoring
 - Security enforcement
 - Professional UI
 - Comprehensive logging
 - Practical cyber security application
-

12. Conclusion

SecureSandbox demonstrates key concepts in:

- Process isolation
- Resource monitoring
- Security enforcement
- Behavioral analysis
- Malware analysis techniques

Learning Outcomes

- Understanding process execution
- OS security mechanisms
- Resource management
- System call monitoring
- Threat detection

Cyber Security Skills

- Sandbox development
- Process isolation
- Security analysis
- Monitoring systems
- Incident response

Project By: [Your Name]

Course: BS Cyber Security

Semester: [Your Semester]

Date: January 2026

Appendix

Dependencies

```
Python 3.7+  
tkinter  
psutil==5.9.0+
```

File Sizes

- Main Program: ~15 KB
- Test Programs: ~1-2 KB each
- Reports: ~10-50 KB (JSON)

Performance

- CPU Overhead: <5%
- Memory Overhead: ~50-100 MB
- Monitoring Interval: 1 second
- Log Retention: Last 50 entries (UI)

End of Documentation